

DIGITAL IMAGE PROCESSING LABORATORY

Semester	B.E. Semester VII
Subject	Digital Image Processing
Subject Professor In-charge	Prof. Rajashree Soman
Laboratory	LO5

Student Name	Amey Pawar	Division	B
Roll Number	23108B0057	Batch	2
Grade and Subject Teacher's Signature			

Experiment Number	7
Experiment Title	Region Growing, Connectivity Comparison, and Region Splitting and Merging
Aim	To implement seeded region growing (with the region mean updated as pixels are absorbed) for segmenting the iris/pupil in grayscale near-infrared eye-crop images; to compare 4-neighbourhood and 8-neighbourhood connectivity quantitatively (region size, shape, boundary behaviour and processing time); and to implement quadtree-based region splitting and merging as a second region-based segmentation technique – continuing the enhancement (Task 3, histogram equalization) → edge/threshold segmentation (Task 6) pipeline on the same 3 dataset images, with quantitative rather than visual-only evaluation throughout. Why the pupil is the region of interest. In every one of these near-IR eye crops the pupil/iris is the darkest, most compact, roughly circular structure against a brighter sclera and skin – exactly the seed-and-grow shape assumption region growing depends on, and its area over time is the geometric signal PERCLOS-style drowsiness detection (<code>src/perclos.py</code>) ultimately consumes: a shrinking or vanishing dark region is the visible eye-closure signal this whole project is built to detect. Figure 20 places this task in the full pipeline: acquisition → enhancement (Task 3) → filtering (Task 4) → segmentation (Task 6 / 7, this report) → feature extraction → drowsiness decision.
Theory & Formal Background	Formal problem statement Let the image domain be a discrete grid $\Omega \subset \mathbb{Z}^2$, with intensity function $I : \Omega \rightarrow \{0, \dots, 255\}$ (the <code>uint8</code> convention used throughout this project). A segmentation of Ω is a partition into n disjoint regions $R_1, \dots, R_n$ satisfying the classical region-based conditions: $\cup_i R_i = \Omega$ – every pixel is assigned to some region; $R_i \cap R_j = \emptyset$ for $i \neq j$ – regions do not overlap; - each R_i is a <i>connected</i> set under the chosen connectivity (4- or 8-neighbour adjacency – Figure 15); $H(R_i) = \text{TRUE}$ for every i – each region individually satisfies a homogeneity predicate H; $H(R_i \cup R_j) = \text{FALSE}$ for every pair of <i>adjacent</i> regions R_i, R_j – the partition is <i>maximal</i>: no two adjacent regions could be merged without breaking homogeneity. Region growing (Algorithm A) builds a single region bottom-up, pixel by pixel, from a seed p_0, testing condition 4 <i>incrementally</i> as each candidate pixel is considered, rather than evaluating it once over a final fixed pixel set (flowchart: Figure 14). Region splitting and merging (Algorithm C) instead starts from the whole domain Ω (which

trivially satisfies conditions 1–3 but not condition 4) and works top-down then bottom-up: **SPLIT** enforces condition 4 by recursively quartering any block that fails it (a quadtree – Figure 17), and **MERGE** enforces condition 5 by recombining adjacent leaves whose union would still be homogeneous (Figure 18).

Page 1 of 20

The homogeneity predicate and quadtree splitting

For splitting, H is implemented as a simple intensity-range test on a block B : $H(B) \equiv \max_{p \in B} I(p) - \min_{p \in B} I(p) \leq \Theta$ (`_homogeneous` in `src/region_growing.py`), with Θ the same `homogeneity_threshold` used for both splitting and (by default) merging. For region growing there is no single $H(R)$ evaluated over the whole region after the fact; growth is governed by a *per-pixel* similarity criterion applied as each candidate is examined – a stronger, order-sensitive proxy for the same idea, detailed next.

The similarity criterion: fixed seed vs. running mean

Two different rules exist for deciding whether a candidate pixel q , adjacent to an already-absorbed pixel p , should join the growing region R :

- **Fixed-seed variant:** admit q iff $|I(q) - I(\text{seed})| \leq T$. The comparison target is fixed at $I(\text{seed})$ for the entire run – every decision is independent of everything the region has grown into so far.
- **Running-mean variant** (implemented here as the default, `region_grow(..., update_mean=True)` – the assignment's "update the region mean if required"): admit q iff $|I(q) - \mu_R| \leq T$, where $\mu_R = (1/|R|) \sum_{r \in R} I(r)$ is *recomputed after every admission*. μ_R starts at $I(\text{seed})$ (when $|R|=1$) and drifts as brighter or darker pixels are absorbed.

`src/region_growing.py`'s `region_grow` implements the running-mean variant by default and exposes `update_mean=False` only as the fixed-seed alternative; every measured result in this report (Figures 4/7/10, the threshold-metrics table, and the 4- vs 8-connectivity comparison in Figures 5/8/11) uses the running-mean variant.

Why this is the report's most interesting technical point. The running mean makes admission *order-dependent*. Whether a given candidate q is admitted depends not just on $I(q)$ and T but on μ_R 's value at the exact moment q is tested – and that value depends on the precise sequence in which earlier pixels were popped from the BFS stack and absorbed. Two BFS traversals that eventually visit the *same* pixel set in a *different order* can therefore make a *different* admit/reject decision for the same borderline pixel, purely because μ_R has drifted to a different value by the time each traversal reaches it. This is exactly the mechanism behind the measured 4- vs 8-connectivity *non-superset* result (Output – "4- vs 8-connected comparison" table): 8-connectivity's BFS visits diagonal neighbours the 4-connected traversal never considers, which changes the order pixels enter the running sum, which shifts μ_R 's trajectory earlier. On `good_lighting` this measurably flips 72 pixels from admitted (4-connected) to rejected (8-connected) even though those 72 pixels are themselves reachable via plain orthogonal steps both traversals share – they are excluded from the 8-connected run purely because its already-shifted running mean fails the $|I(q) - \mu_R| \leq T$ test for them by the time it is their turn, not because 8-connectivity could not reach them. A fixed-seed criterion could never produce this asymmetry: since $I(\text{seed})$ never changes, 8-connectivity is *guaranteed* to be a pixel-for-pixel superset of 4-connectivity under it (extra diagonal paths can only add reachable pixels, never revoke a decision already fixed by a constant reference value). The running mean trades that clean guarantee for a region estimate that adapts to what has actually been absorbed so far – generally desirable, since a fixed seed value is a single noisy sample while the running mean is a self-correcting estimate of "typical pupil intensity so far" – but it is the direct, provable cause of the measured non-superset behaviour, not an implementation bug.

Pseudocode: region growing (BFS)

```

1  procedure REGION_GROW(I, seed, T, N)           # N = 4- or 8-neighbour
   offset set
2      visited[seed] ← true; mask[seed] ← 255
3      sum ← I[seed]; count ← 1; μ ← sum / count
4      push(S, seed)
```

```

5   while S not empty do
6       p ← pop(S)
7       for each q in N(p) do
8           if not visited[q] then
9               visited[q] ← true
10              if |I[q] - μ| ≤ T then
11                  mask[q] ← 255
12                  sum ← sum + I[q]; count ← count + 1
13                  μ ← sum / count          # running-mean update,
every admission
14                  push(S, q)
15   return mask, count

```

Page 2 of 20

Pseudocode: quadtree splitting

```

1   procedure SPLIT(I, r0, r1, c0, c1, minSize, θ, leaves)
2       B ← I[r0:r1, c0:c1]
3       if (r1-r0) ≤ minSize or (c1-c0) ≤ minSize or HOMOGENEOUS(B, θ)
then
4           leaves ← leaves ∪ {(r0,r1,c0,c1)}
5           return
6       rm ← r0 + (r1-r0)/2; cm ← c0 + (c1-c0)/2
7       SPLIT(I, r0, rm, c0, cm, minSize, θ, leaves)      # top-left
8       SPLIT(I, rm, r1, c0, cm, minSize, θ, leaves)      # top-right
9       SPLIT(I, r0, rm, cm, c1, minSize, θ, leaves)      # bottom-left
10      SPLIT(I, rm, r1, cm, c1, minSize, θ, leaves)      # bottom-right
11
12  function HOMOGENEOUS(B, θ)
13      return max(B) - min(B) ≤ θ

```

Pseudocode: region merging (union-find)

```

1   procedure MERGE(leaves, τ)                # τ = merge_tolerance
2       for i ← 1 to L do parent[i] ← i      # union-find init, one
leaf per set
3       μi ← mean(leaves[i]) for every i
4       for i ← 1 to L do
5           for j ← i+1 to L do
6               if ADJACENT(leaves[i], leaves[j]) and |μi - μj| ≤ τ then
7                   UNION(parent, i, j)
8       for i ← 1 to L do
9           labels[leaves[i]] ← FIND(parent, i)
10      return labels
11
12  function FIND(parent, x)                    # path compression
13      while parent[x] ≠ x do
14          parent[x] ← parent[parent[x]]
15          x ← parent[x]
16      return x
17
18  procedure UNION(parent, x, y)
19      rx ← FIND(parent, x); ry ← FIND(parent, y)
20      if rx ≠ ry then parent[rx] ← ry

```

Complexity analysis

Region growing (A). Every pixel is pushed onto the stack at most once, guarded by the visited array set the first time it is examined as a neighbour (pseudocode lines 8–9), so the outer loop body runs $O(N)$ times for an N -pixel image. Each popped pixel performs a constant number of neighbour tests (4 or 8), each $O(1)$ – a comparison, at most one array write, and an $O(1)$ incremental mean update. **Total time $O(N)$; space $O(N)$** for the visited/mask arrays plus $O(N)$ worst case for the stack.

Quadtree splitting (C, SPLIT). At each recursion depth d , the current set of blocks exactly partitions the image, so the pixels examined by the homogeneity test (a max/min scan) summed across all blocks at that depth is $O(N)$, regardless of block count. The

recursion halves each block's side length per level, reaching the `min_size` floor after $O(\log N)$ levels. **Total time $O(N \log N)$; space $O(N)$** for leaves plus $O(\log N)$ recursion depth.

Page 3 of 20

Merging (C, MERGE). With L leaves, the union-find operations themselves (FIND with path compression, UNION) are $O(\alpha(L))$ amortized per call – effectively linear, i.e. **$O(L\alpha(L))$** total for the unions actually performed. As implemented, however, the adjacency test that decides *which pairs to attempt* is a plain nested loop over all leaf pairs (pseudocode lines 4–7): $O(L^2)$ pair tests, each $O(1)$. Since L is a few hundred here (Output – "Split-merge" table) against N in the tens of thousands, $O(L^2)$ stays fast in absolute terms ($L^2 \approx 62,500$ for $L=250$) – but the $O(L\alpha(L))$ bound is precisely for the union-find machinery, not the adjacency-pair enumeration wrapped around it; a spatially-indexed adjacency lookup would bring the whole merge step down to $O(L \log L)$ or better, and matters more as `min_size` shrinks and L grows (Limitations & Future Work).

Relating this to the measured timings. Both connectivity variants of `region_grow` are the *same* $O(N)$ algorithm; the constant factor differs only because 8-connectivity performs exactly twice as many neighbour tests per popped pixel (8 vs 4). The measured ratio ($5.68/3.90 \approx 1.45\times$, Output – "4- vs 8-connected" table) is well under the naive $2\times$ that test count alone would predict – explained next.

Why 8-connectivity costs $1.45\times$, not $2\times$

Doubling the neighbour-test count per popped pixel does not double wall-clock time, for two compounding reasons visible in this implementation:

- 1. Fixed per-pixel overhead.** Popping from the deque, writing to `visited`/`mask`, and recomputing the running mean happen once per popped pixel *regardless* of connectivity – this fixed cost does not scale with neighbour count, diluting the $2\times$ variable cost toward a smaller overall ratio.
- 2. Memory locality and early rejection.** The four extra diagonal offsets $((-1,-1), (-1,+1), (+1,-1), (+1,+1))$ address pixels in the row immediately above/below the current row, already resident in cache from processing that row's orthogonal neighbours moments earlier – so the "extra" lookups mostly hit warm cache lines rather than causing additional misses, and any candidate that has already been absorbed via a cheaper orthogonal path short-circuits at the `visited` check before the $|I(q) - \mu_R| \leq T$ test ever runs.

The net effect measured here ($1.42\times$ – $1.47\times$ across the 3 images, mean $1.45\times$, Output – "4- vs 8-connected" table) sits well below the naive $2\times$ prediction from raw test count alone.

Resources / Apparatus Required

Software: Python 3.11 • OpenCV • NumPy • Matplotlib • scikit-image • Pillow • Jupyter Notebook / VS Code

Algorithm

A. Region Growing

1. Start the program.
2. Read the enhanced (histogram-equalized) grayscale eye-crop image as a NumPy `uint8` array (same enhancement as Task 6's input, for pipeline continuity).
3. Compute an **automatic seed**: Gaussian-blur the image (5×5) to suppress single-pixel noise, then take the coordinate of the blurred image's global minimum (darkest point) – this reliably lands inside the pupil in all 3 test images.
4. Also record a **manual seed**: the geometric centre of the crop, for comparison.
5. From the seed, run BFS: maintain a running region mean (initially the seed's own intensity) and a visited set; for every unvisited 4- or 8-neighbour of a pixel just absorbed, admit it into the region if $\text{abs}(\text{neighbour} - \text{running_mean}) \leq T$.

Page 4 of 20

6. **Update the region mean** after every pixel is absorbed (running mean over all pixels absorbed so far), so later admission decisions track the region's own evolving statistics rather than only the original seed value.
7. Repeat until the queue is empty; output the binary region mask and its pixel count.
8. Repeat for 3 thresholds (Low, Medium, High) and, for the metrics table, a 4th (Very High) to locate where the region starts leaking past the expected pupil area.
9. Stop the program.

B. 4- vs 8-Connectivity

1. Start the program.
2. Using the same auto seed and a fixed threshold (Medium, $T=25$), run region growing twice on the same image: once considering only the 4 orthogonal neighbours (up/down/left/right), once considering all 8 neighbours (orthogonal + 4 diagonals).
3. Compute the pixel-count difference, the symmetric difference (XOR) mask between the two results, and each mask's shape compactness ($4 \times \pi \times \text{Area} / \text{Perimeter}^2$ on the largest external contour, via `cv2.findContours / cv2.contourArea / cv2.arcLength`).
4. Time both variants over 30 repeated calls on the same image/seed/threshold and report the mean per-call time in milliseconds.
5. Stop the program.

C. Region Splitting and Merging

1. Start the program.
2. Recursively **split** the image as a quadtree: starting from the whole frame, test a block's homogeneity as `max(block) - min(block) <= threshold`; if homogeneous, or the block has hit the minimum block size, keep it as a leaf; otherwise split it into 4 quadrants (top-left/top-right/bottom-left/bottom-right) and recurse on each.
3. Compute each leaf's mean intensity.
4. **Merge**: build an adjacency graph over leaves that share a boundary edge; union two adjacent leaves (via union-find) whenever their mean intensities differ by no more than the same tolerance used for splitting.
5. Relabel every leaf by its union-find root to produce the final merged-region label map.
6. Report the number of leaves after splitting and the number of regions remaining after merging.
7. Stop the program.

Formal, line-numbered pseudocode for all three procedures above, a flowchart (Figure 14) for Algorithm A, and a full time/space complexity analysis are given in **Theory & Formal Background** above.

Program code

Trimmed to the essential logic. Full module: `src/region_growing.py`. Every function takes/returns a single-channel uint8 NumPy array; nothing in `src/enhancement.py` or `src/segment.py` (both reused as-is) is modified.

A. Region Growing

```
def auto_seed(img, blur_ksize=5):
    blurred = cv2.GaussianBlur(img, (blur_ksize, blur_ksize), 0)
    idx = int(np.argmax(blurred))
    return np.unravel_index(idx, blurred.shape) # (row, col)

def region_grow(img, seed, T, connectivity=8, update_mean=True):
    neighbours = CONN8 if connectivity == 8 else CONN4
    visited = np.zeros(img.shape, dtype=bool); mask =
np.zeros(img.shape, dtype=np.uint8)
    sr, sc = seed
    visited[sr, sc] = True; mask[sr, sc] = 255
    region_sum, region_count = float(img[sr, sc]), 1
```

```

mean = region_sum / region_count
q = deque([(sr, sc)])
while q:
    r, c = q.popleft()
    for dr, dc in neighbours:
        nr, nc = r + dr, c + dc
        if 0 <= nr < h and 0 <= nc < w and not visited[nr, nc]:
            visited[nr, nc] = True
            if abs(int(img[nr, nc]) - mean) <= T:
                mask[nr, nc] = 255
                region_count += 1; region_sum += img[nr, nc]
                if update_mean: # "update the region
                    mean = region_sum / region_count
            q.append((nr, nc))
return mask, region_count

```

Page 5 of 20

C. Region Splitting and Merging

```

def _homogeneous(block, threshold):
    return int(block.max()) - int(block.min()) <= threshold

def _quadtree_split(img, r0, r1, c0, c1, min_size, threshold, leaves):
    h, w = r1 - r0, c1 - c0
    if h <= min_size or w <= min_size or _homogeneous(img[r0:r1, c0:c1],
threshold):
        leaves.append((r0, r1, c0, c1)); return
    rm, cm = r0 + h // 2, c0 + w // 2
    _quadtree_split(img, r0, rm, c0, cm, min_size, threshold, leaves) #
top-left
    _quadtree_split(img, r0, rm, cm, c1, min_size, threshold, leaves) #
top-right
    _quadtree_split(img, rm, r1, c0, cm, min_size, threshold, leaves) #
bottom-left
    _quadtree_split(img, rm, r1, cm, c1, min_size, threshold, leaves) #
bottom-right

def split_merge(img, min_size=20, homogeneity_threshold=25,
merge_tolerance=None):
    leaves = []; _quadtree_split(img, 0, h, 0, w, min_size,
homogeneity_threshold, leaves)
    means = [float(img[r0:r1, c0:c1].mean()) for (r0,r1,c0,c1) in
leaves]
    parent = list(range(len(leaves))) # union-find
    for i, j in adjacent_leaf_pairs(leaves):
        if abs(means[i] - means[j]) <= (merge_tolerance or
homogeneity_threshold):
            union(parent, i, j)
    return relabel_by_root(leaves, parent) # -> labels,
n_leaves, n_regions

```

Output

Same 3 test images as Task 3 / Task 6, each loaded, upscaled with `cv2.INTER_NEAREST` (minimum 300px side), and histogram-equalized before region growing / splitting / merging.

Page 6 of 20

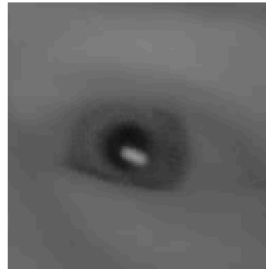
Label	File	Condition	Native size	Native min / max intensit
-------	------	-----------	-------------	---------------------------

good_lighting	alert/s0031_00366_1_0_1_0_1_02.png	open eye, no glasses, good lighting, no reflection	128x128	19 / 148
poor_lighting	alert/s0014_08297_0_0_1_1_0_02.png	open eye, no glasses, poor/uneven lighting	130x130	8 / 255
glasses_reflection	alert/s0035_00133_0_1_1_2_1_02.png	open eye, eyeglasses + strong IR reflection (glare hotspot)	144x144	27 / 255

Step 1 – original / grayscale, with dimensions and intensity range

Figure 1: Step 1 – good_lighting: 128x128 native, min=19, max=148 (source file is already single-channel IR, so panels are numerically identical)

Original (as loaded, IMREAD_COLOR)



Grayscale (cvtColor_BGR2GRAY)

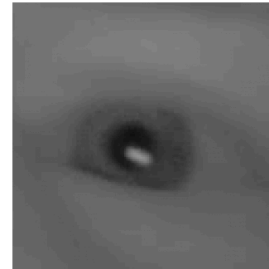


Figure 1: Step 1 – good_lighting: 128x128, min=19, max=148.

Figure 2: Step 1 – poor_lighting: 130x130 native, min=8, max=255 (source file is already single-channel IR, so panels are numerically identical)

Original (as loaded, IMREAD_COLOR)



Grayscale (cvtColor_BGR2GRAY)

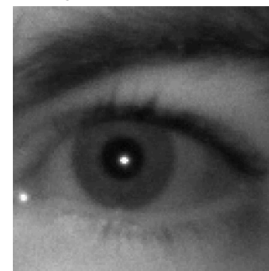
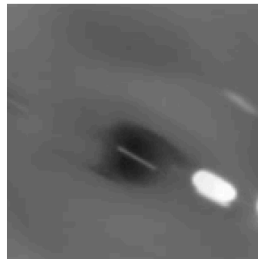


Figure 2: Step 1 – poor_lighting: 130x130, min=8, max=255.

Page 7 of 20

Figure 3: Step 1 – glasses_reflection: 144x144 native, min=27, max=255 (source file is already single-channel IR, so panels are numerically identical)

Original (as loaded, IMREAD_COLOR)



Grayscale (cvtColor_BGR2GRAY)

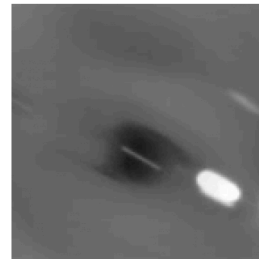


Figure 3: Step 1 – glasses_reflection: 144x144, min=27, max=255.

The dataset PNGs are already single-channel infrared crops, so the "Original" (IMREAD_COLOR, channel-replicated) and "Grayscale" (cvtColor_BGR2GRAY) panels are numerically identical here – both panels are still produced to follow the standard Step-1 read/convert/report procedure.

Seeds used

Image	Auto seed (row, col)	Auto seed intensity	Manual seed (row, col)	Manual seed intensity
good_lighting	(123, 114)	0	(128, 128) – centre	1
poor_lighting	(133, 106)	0	(130, 130) – centre	54
glasses_reflection	(152, 143)	0	(144, 144) – centre	3

The automatic seed (darkest point after a 5x5 Gaussian blur) lands exactly on a 0-intensity pixel inside the pupil on all 3 images. The manual seed (geometric centre of the crop) is close by construction but noticeably brighter on *poor_lighting* (intensity 54 vs. 0) – the pupil there is off-centre in the crop, so a naive "assume the pupil is centred" manual placement is measurably worse than the automatic darkest-point seed.

Region growing at 3 thresholds, per image

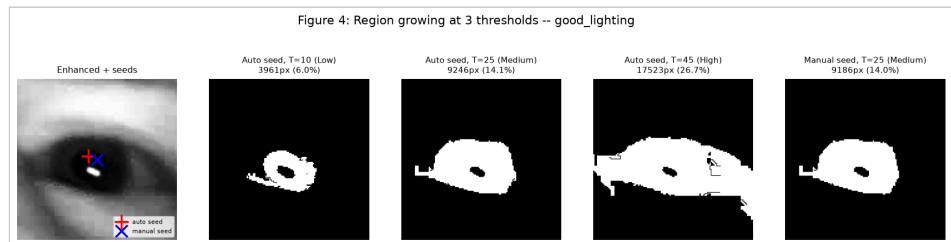


Figure 4: Region growing, auto seed at Low/Medium/High T, and manual seed at Medium T – good_lighting.

Page 8 of 20

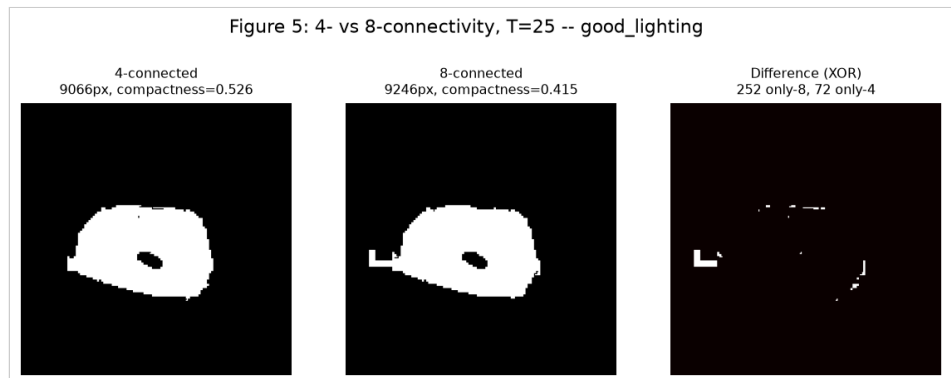


Figure 5: 4- vs 8-connectivity at T=25, with XOR difference map – good_lighting.

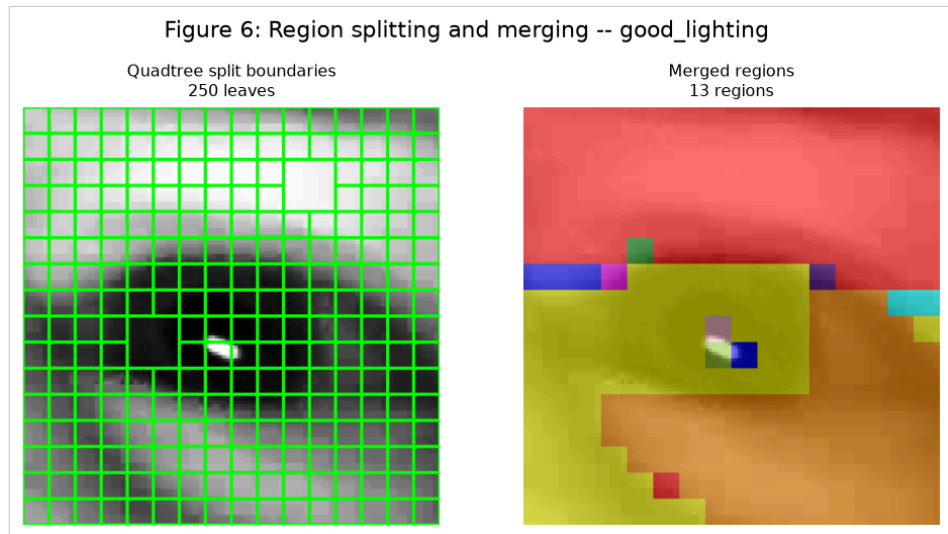


Figure 6: Quadtree split boundaries and final merged regions – good_lighting.

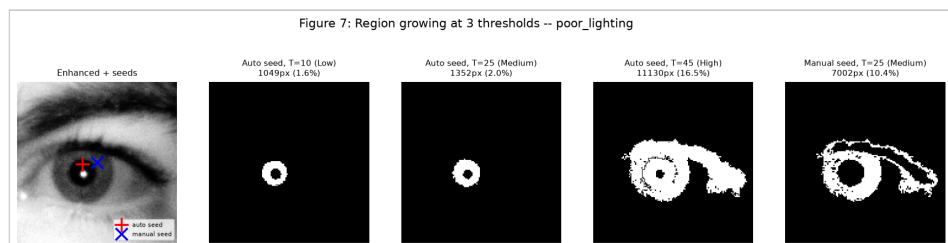


Figure 7: Region growing, auto seed at Low/Medium/High T, and manual seed at Medium T – poor_lighting.

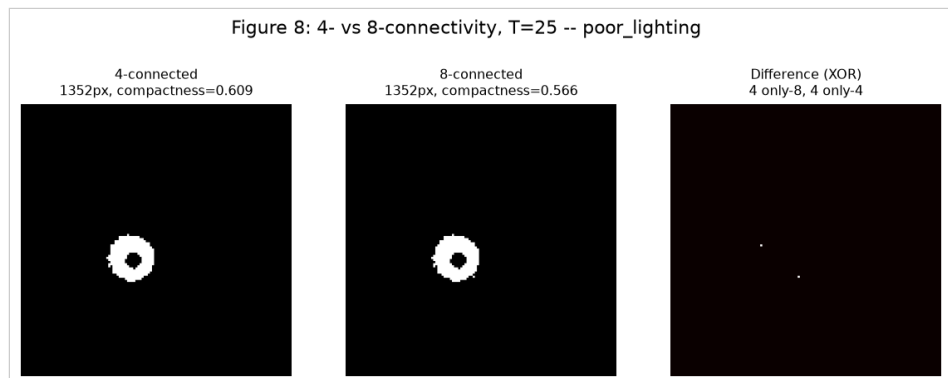


Figure 8: 4- vs 8-connectivity at T=25, with XOR difference map – poor_lighting.

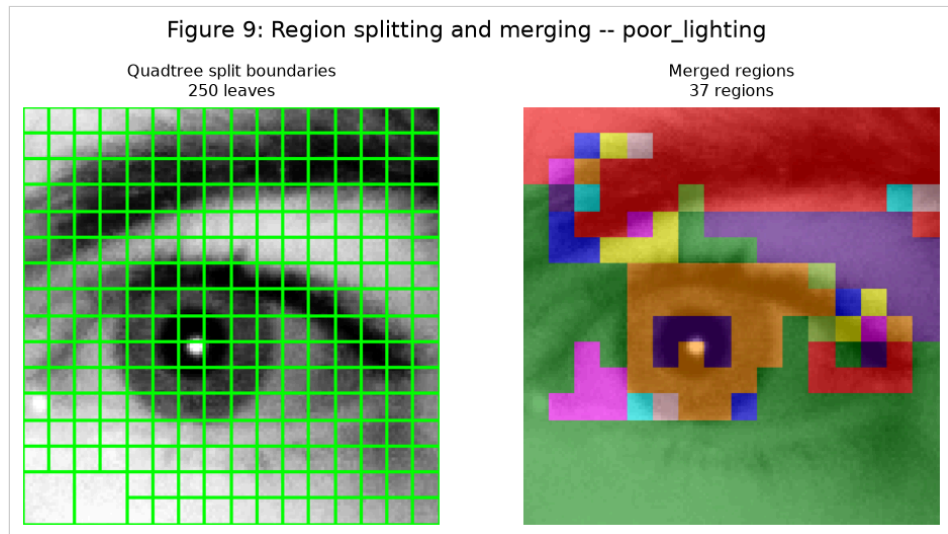


Figure 9: Quadtree split boundaries and final merged regions – poor_lighting.

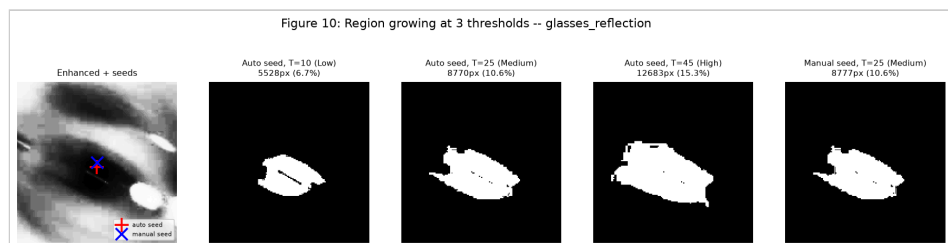


Figure 10: Region growing, auto seed at Low/Medium/High T, and manual seed at Medium T – glasses_reflection.

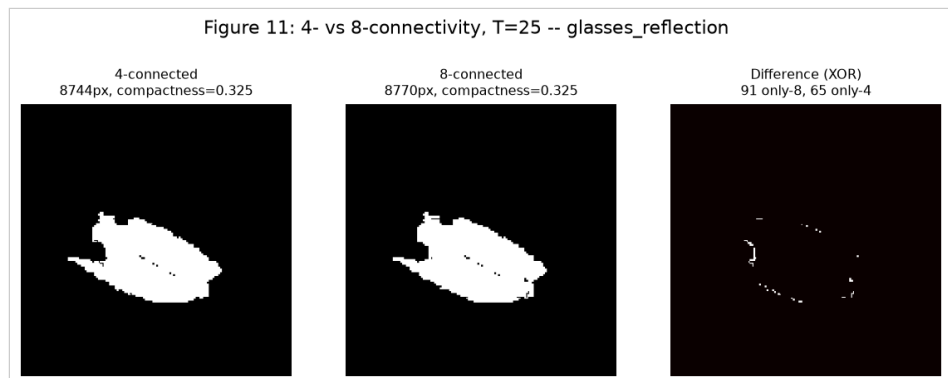


Figure 11: 4- vs 8-connectivity at T=25, with XOR difference map – glasses_reflection.

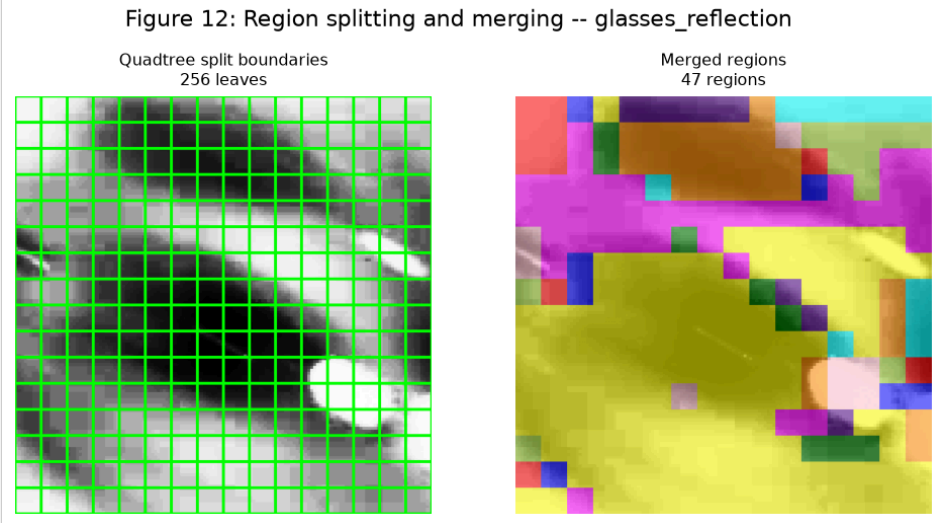


Figure 12: Quadtree split boundaries and final merged regions – glasses_reflection.

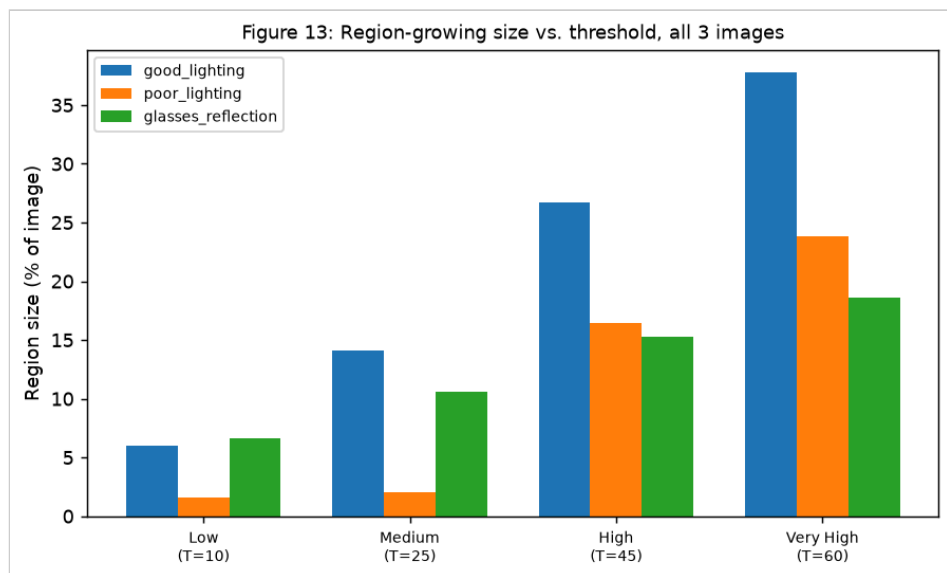
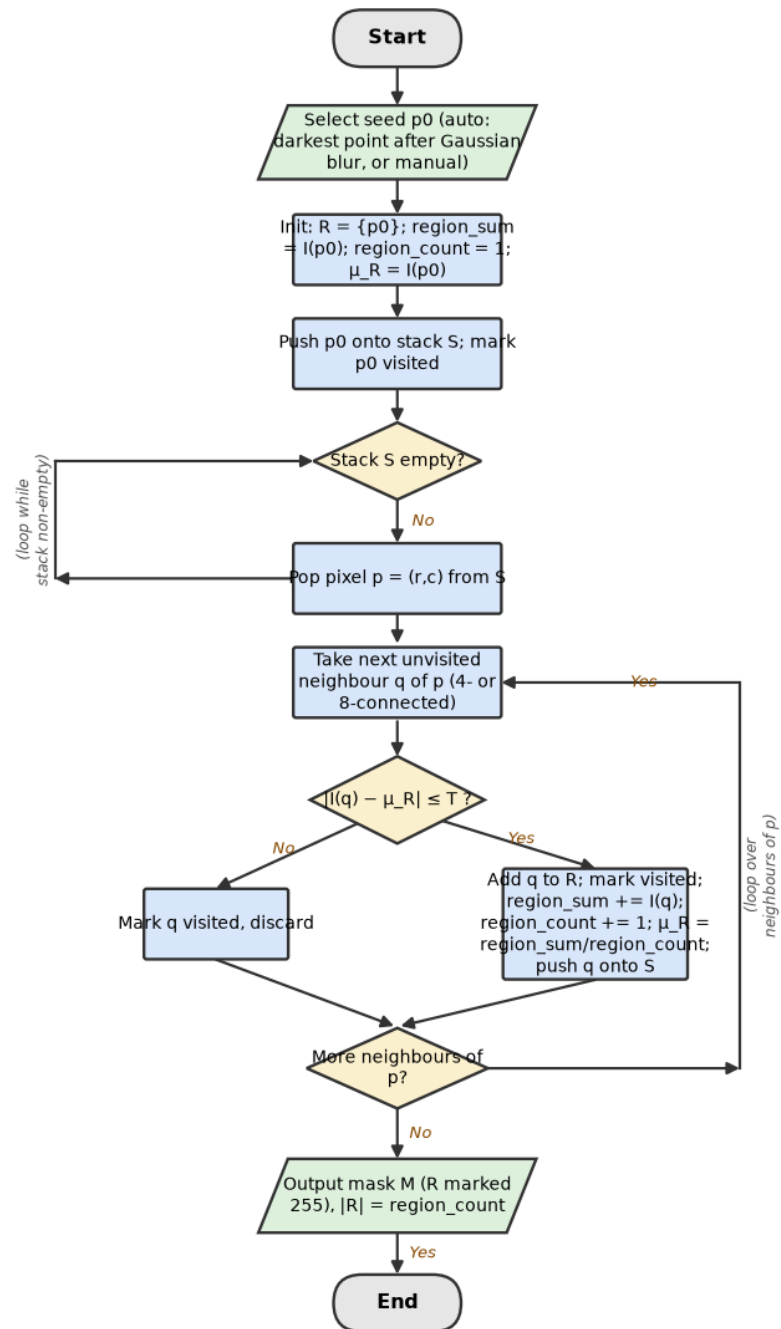


Figure 13: Region size (% of image) vs. threshold, all 3 images.

Explanatory diagrams (Figures 14–20)

The 13 figures above report *measured* results. The 7 figures below are explanatory diagrams supporting the formal treatment in Theory & Formal Background and the failure-mode analysis in Analysis & Discussion – schematics and a flowchart for the algorithms themselves, plus a scanline diagnostic built from the same 3 dataset images.

Figure 14: Region-growing flowchart -- BFS from seed p_0 , admitting neighbour q when $|I(q) - \mu_R| \leq T$, with μ_R the running region mean updated after every admission

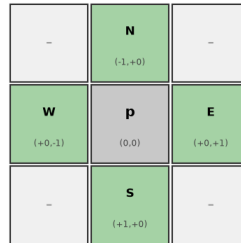


- Terminator (start / end)
- Input / Output
- Process
- Decision

Figure 14: Region-growing flowchart (Algorithm A) – BFS from seed p_0 , admitting neighbour q when $|I(q) - \mu_R| \leq T$, with μ_R the running region mean updated after every admission. Compare against the pseudocode in Theory & Formal Background.

Figure 15: Pixel-neighbourhood definitions used by `region_grow(...,connectivity=4|8)`. Each cell is labelled with its ($\Delta\text{row},\Delta\text{col}$) offset from the centre pixel p ; green = admitted as a candidate neighbour, grey/dash = not considered a neighbour.

4-connected neighbourhood $N_4(p)$
(orthogonal only)



8-connected neighbourhood $N_8(p)$
(orthogonal + diagonal)

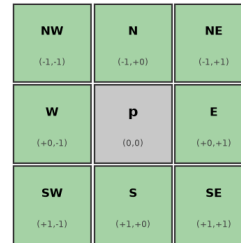


Figure 15: 4-connected vs. 8-connected pixel-neighbourhood definitions used by `region_grow(..., connectivity=4|8)`, with each cell's ($\Delta\text{row},\Delta\text{col}$) offset from the centre pixel p labelled.

Figure 16: Region-growing progression – `good_lighting`, auto seed, $T=45$ (High). The frontier expands outward from the seed (yellow +) one BFS layer at a time; final region = 17523px.

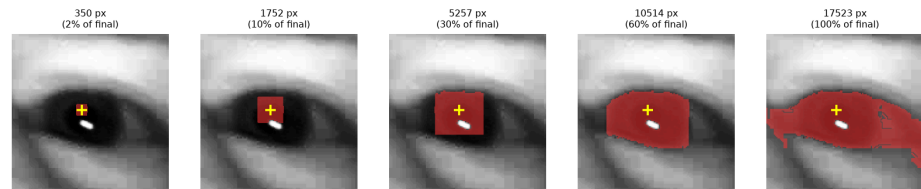


Figure 16: Region-growing progression on `good_lighting` (auto seed, $T=45$ /High) – the frontier expands outward from the seed one BFS layer at a time, shown at 2%, 10%, 30%, 60% and 100% of the final region size.

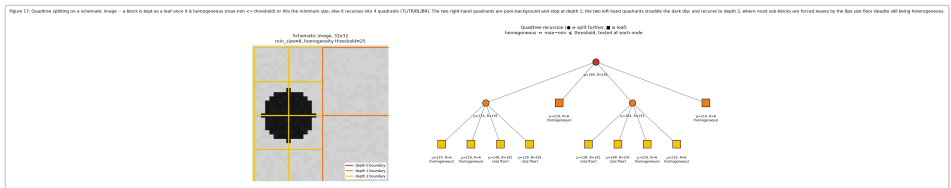


Figure 17: Quadtree splitting (Algorithm C, SPLIT) on a schematic 32x32 image – recursive quadrant boundaries (left) and the corresponding recursion tree (right), annotated with each node's mean, intensity range, and the reason it became a leaf ("homogeneous" vs. "size floor").

Figure 18: Region adjacency graph and the merge criterion, on a schematic 4-cluster image. 16 adjacent leaf pairs merge (mean difference within tolerance 20), 8 stay separate (cluster boundaries) – 16 leaves collapse to 4 regions.

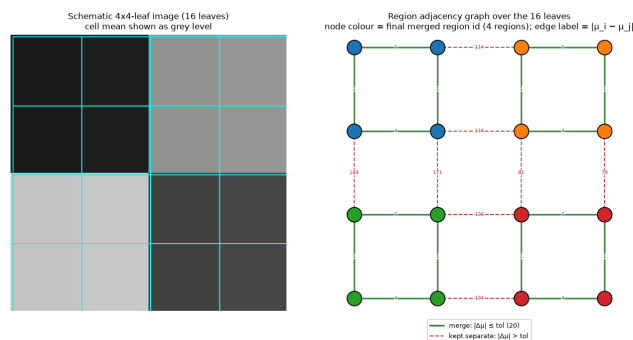


Figure 18: Region adjacency graph and the merge criterion (Algorithm C, MERGE) on a schematic 4-cluster image – nodes are quadtree leaves coloured by their final merged region id (4 regions); edges are leaf adjacency annotated with $|\mu_i - \mu_j|$, solid green where that difference is within the merge tolerance (merge) and dashed red where it is not (kept separate).

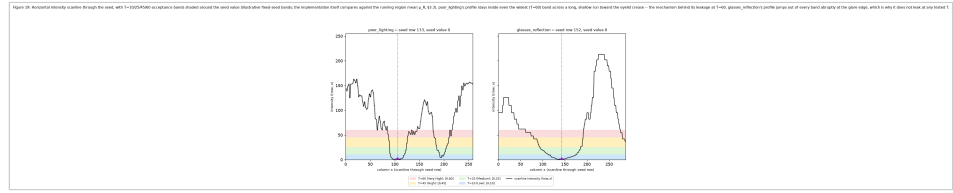


Figure 19: Horizontal intensity scanline through the seed pixel, **poor_lighting** vs. **glasses_reflection**, with illustrative T=10/25/45/60 acceptance bands shaded around the seed value. Referenced throughout the failure-mode analysis below.

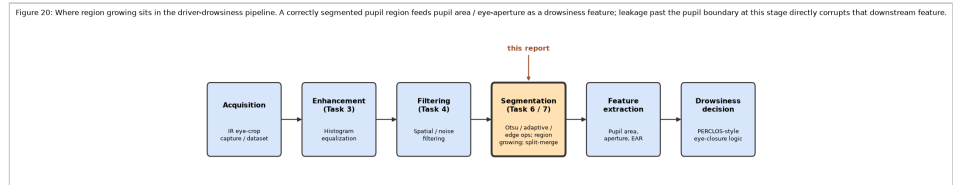


Figure 20: Where region growing and split-merge sit in the overall driver-drowsiness pipeline – acquisition → enhancement (Task 3) → filtering (Task 4) → segmentation (Task 6/7) → feature extraction → drowsiness decision.

Threshold effect (region size, missing pixels, leakage, segmentation accuracy)

"Missing pixels" / "leakage" / "segmentation accuracy" are measured against a reference pupil mask reused as-is from `src.segment.segment_eye` (Otsu value scaled by 0.7 to anchor on the pupil's darker core, then largest-connected-component isolation) – not a hand-labelled ground truth, but the same reference Task 6 documents as this project's own pupil-isolation anchor. Missing = reference pixels the grown region has not yet reached; leakage = grown-region pixels outside that reference; accuracy = IoU between the two.

Image	Thresh old	T	Region size (px, % image)	Reference pupil px	Missing px	Leakage px	Accuracy (IoU)
good_lighting	Low	10	3,961 (6.0%)	22,316	18,355	0	0.178
	Medium	25	9,246 (14.1%)		13,070	0	0.414
	High	45	17,523 (26.7%)		4,793	0	0.785
	Very High	60	24,750 (37.8%)		429	2,863	0.869
poor_lighting	Low	10	1,049 (1.6%)	11,880	10,831	0	0.088
	Medium	25	1,352 (2.0%)		10,528	0	0.114
	High	45	11,130 (16.5%)		750	0	0.937
	Very High	60	16,134 (23.9%)		21	4,275	0.734
glasses_reflection	Low	10	5,528 (6.7%)	15,776	10,248	0	0.350
	Medium	25	8,770 (10.6%)		7,006	0	0.556
	High	45	12,683 (15.3%)		3,093	0	0.804
	Very High	60	15,404 (18.6%)		372	0	0.976

4- vs 8-connected comparison, T=25 (Medium)

Page 14 of 20

Connectivity	Region size (px)	Shape (compactness, mean)	Connectivity of diagonal pixels	Region leakage (extra px from diagonal steps)	Processing time (mean, ms/call)	Suitability for this application

4- connected	9,066 / 1,352 / 8,744	0.487 (rounder)	Excluded – only orthogonal steps	o (reference)	3.90	Cheaper, but misses diagonal- adjacent pupil- boundary pixels
8- connecte d	9,246 / 1,352 / 8,770	0.435 (more jagged)	Included – orthogonal + 4 diagonals	+252 / +4 / +91 px (up to 2.8%)	5.68 (1.45x slower)	Recommen ded default – absorbs real diagonal- adjacent pupil pixels that 4- conn's orthogonal- only steps miss; ~46% slower per call is immaterial at <10ms/call on these crop sizes

Region size triplets are (good_lighting / poor_lighting / glasses_reflection). On poor_lighting the two connectivities give an **identical** pixel count (1,352) – the small, tightly-bounded pupil ring there never needs a diagonal-only step to reach any of its neighbours. Neither connectivity is strictly a superset of the other: on good_lighting, 252 pixels are reachable only via 8-connectivity's diagonal steps, but 72 pixels are absorbed by 4-connectivity *and not* 8-connectivity – because 8-connectivity's diagonal-first paths shift the running region mean earlier in the BFS, so a handful of borderline pixels that 4-connectivity's slower orthogonal-only mean drift still admits fail the 8-connected run's already-shifted mean test. Full per-image numbers in results/task7_connectivity_metrics.csv.

Split-merge (quadtree split, min block 20px, homogeneity/merge tolerance 25)

Image	Leaves after split	Regions after merge	Reduction
good_lighting	250	13	19.2x
poor_lighting	250	37	6.8x
glasses_reflection	256	47	5.4x

Analysis & Discussion

The measured trends in Output follow directly from the mechanisms formalised in Theory & Formal Background. Region size grows with T because a larger T widens the running-mean acceptance window, and the growth is non-linear because that window's effect compounds with the region's own shape: while the frontier is still inside the dark, fairly uniform pupil core (visible as the tight red patches in Figure 16's early snapshots), μ_R barely moves and growth is slow; once the frontier reaches a boundary the current T can cross, growth accelerates sharply (Figure 16's 60%→100% panels) – this is the same mechanism as the Medium→High jump documented for poor_lighting in Observations below. The 4- vs 8-connectivity divergence is a direct, provable consequence of the running-mean criterion being order-dependent (Theory & Formal Background – "The similarity criterion"), not measurement noise. Split-merge's very different leaf-reduction ratios across the 3 images (19.2x on good_lighting vs. 5.4x on glasses_reflection) reflect how many genuinely distinct intensity plateaus each image's illumination pattern creates for the quadtree's homogeneity test to separate – Figure 17 shows the same mechanism in miniature on a controlled schematic image, where only the quadrants that straddle a real boundary need to recurse past depth 1.

Failure-mode analysis

Why poor_lighting leaks at T=60. Figure 19 (left panel) plots the intensity scanline through this image's seed row. The pupil-to-eyelid-crease transition is a *long, shallow ramp*: the profile stays inside even the widest shaded band (T=60) across a wide run of columns either side of the seed, rather than crossing it in a sharp jump. Because the running mean μ_R only ever moves gradually, one admitted pixel at a time (Theory & Formal Background), once growth reaches this ramp it keeps admitting the next, only-slightly-brighter ring of pixels, walking μ_R upward step by step rather than being stopped by any single sharp intensity jump. This is exactly the leakage measured in the threshold-metrics table: missing pixels keep falling (750→21px) but leakage jumps from 0 to 4,275px between High and Very High, which is also why IoU peaks at High (0.937) and falls back at Very High (0.734) for this image alone – the ramp both Figure 19 and this leakage number describe is the same physical feature.

Why glasses_reflection never leaks. Figure 19 (right panel) shows the opposite profile shape for the same seed-row scanline: an abrupt, large jump at the IR glare boundary – the profile leaves every shaded band within a handful of columns rather than drifting through them. No matter how far μ_R has drifted by the time growth reaches that edge, the intensity jump there is far larger than any tested T (up to 60), so the boundary acts as a hard stopping edge the running-mean criterion cannot cross in either direction – consistent with the measured opx leakage at *every* tested threshold including Very High (threshold-metrics table) and visually with Figure 10/11's mask hugging the eyelid contour rather than spilling past the glare.

Application relevance

A correctly segmented pupil region is not an end in itself here – it is the direct input to the geometric features (`src/features.py`, `src/perclos.py`) that PERCLOS-style drowsiness detection consumes downstream (Figure 20): pupil/eye-opening area, and, via a fitted ellipse or bounding box, an aperture proxy, computed frame by frame. A shrinking or vanishing measured area is the actual signal a PERCLOS-style detector watches for eye closure. Leakage at the segmentation stage corrupts this signal in a specific, predictable direction: because leakage only ever *grows* the measured region (no threshold-metrics row shows negative leakage), an eye that is not actually more open would be scored as if it were, biasing the downstream openness proxy upward and *delaying* detection of a closing eye – exactly the failure mode that matters most for a drowsiness detector, where a late detection is the costly error. This is why this report tracks leakage onset as carefully as raw region size: for this application, a slightly undergrown but leakage-free region (`good_lighting` at High, T=45: IoU 0.785, 0 leakage) is arguably a *safer* operating point than a larger but leaking one, even though the larger region's raw IoU against the static reference mask is nominally higher (Very High: IoU 0.869, 2,863px leakage) – accuracy against a static reference mask and safety for a temporal drowsiness signal are not the same criterion.

Comparison with Other Segmentation Techniques

Task 6 evaluated Otsu global thresholding, adaptive thresholding, and Sobel/Laplacian edge operators on the same 3 dataset images. The table below compares all of Task 6's methods against this task's two region-based techniques along the dimensions most relevant to segmenting a pupil under variable IR illumination.

Page 16 of 20

Method	Seed dependence	Parameter sensitivity	Connectivity awareness	Non-uniform illumination handling
--------	-----------------	-----------------------	------------------------	-----------------------------------

	Region growing (this task) High – anchored to one starting pixel; auto darkest-point seeding mitigates but does not remove this (manual seed at intensity 54 on poor_lighting gave a >5x larger Medium-T region, Observations).	High – a single global T controls both reach and leakage onset (3,961→24,750px across T=10→60 on good_lighting).	Explicit and tunable (4 vs. 8, Figure 15) – the only method here where connectivity is itself a first-class parameter.	Adapts locally via the running mean (Theory & Formal Background), tracking a spatially-varying pupil intensity better than a single global cutoff – but the same adaptivity is what lets it drift and leak once a boundary is only a shallow gradient away (Failure-mode analysis above).
	Region split-merge (this task) None – starts from the whole frame, no seed pixel.	High – the homogeneity/merge tolerance controls both leaf count and final region count (250–256 leaves collapsing 5.4x–19.2x, Output).	Implicit only – quadtree leaves are always axis-aligned rectangles; merge adjacency is edge-touching, not pixel-level 4-/8-connectivity.	Locally adaptive per block (each leaf's homogeneity is tested independently of the rest of the image) but boundaries stay block-aligned regardless (Limitations & Future Work).
	Otsu global threshold (Task 6) None.	Low – the threshold is computed automatically from the histogram, no user parameter to tune.	None at the thresholding step itself (a per-pixel binary decision); any connectivity enters only via a separate connected-component pass afterward.	Poor – a single global threshold cannot track spatially-varying illumination; Task 6 documents this as a source of eyebrow/eyelash inclusion in the same Otsu-anchored mask this report reuses as its own reference.
	Adaptive threshold (Task 6) None.	Moderate – block size and constant offset must both be chosen.	None at the thresholding step.	Good by design – the whole point of a locally-computed threshold (mean/Gaussian neighbourhood) is to track slow illumination gradients that defeat a single global Otsu value.
	Edge operators – Sobel/Laplacian (Task 6) None.	Moderate-high – kernel size and the post-hoc binarization threshold on gradient magnitude both matter.	None directly – edge maps mark boundaries, not regions; turning them into a region needs a separate closing/filling step.	Poor to moderate – gradient magnitude responds to any local contrast change, including illumination gradients unrelated to the pupil boundary, producing spurious edges.
	Region growing is the only method in this comparison where connectivity is a tunable, first-class parameter (Figure 15) rather than an afterthought applied to someone else's binary mask – but that same design choice (a running-mean criterion evaluated pixel by pixel) is what makes it seed- and threshold-sensitive in a way Otsu and adaptive thresholding, which have no seed at all, structurally cannot be.			
Observations	Region growing and thresholds. Region size grows monotonically with T on all 3 images, but not linearly: on poor_lighting the region barely moves from Low to Medium (1,049 → 1,352px, a 1.3x step for a 2.5x change in T) and then jumps 8.2x from Medium to High (1,352 → 11,130px) – the pupil ring stays isolated until T crosses the intensity gap to the surrounding dark eyelid crease, at which point growth escapes the ring almost immediately (visible in Figure 7). good_lighting and glasses_reflection			

grow more smoothly (largest single-step ratios 2.33x and 1.59x) because their pupil-to-surround intensity gap is shallower. **Leakage past the reference pupil mask is zero at Low/Medium/High on every image** and only appears at Very High ($T=60$): 2,863px on good_lighting, 4,275px on poor_lighting, but still 0px on glasses_reflection – the IR glare hotspot's sharp bright/dark boundary in that image acts as a natural stopping edge that constrains growth even at the highest threshold tested, exactly the boundary-preserving behaviour Figure 10 shows (the mask hugs the eyelid contour without spilling past it at any of the 4 thresholds). Leakage does not automatically hurt accuracy: on good_lighting and glasses_reflection, IoU keeps rising through Very High (0.785→0.869 and 0.804→0.976) because missing pixels keep shrinking faster than leakage grows, but on poor_lighting IoU actually peaks at High (0.937) and falls back to 0.734 at Very High – there, leakage growth (0→4,275px) outpaces the remaining recovery of missing pixels (750→21px), so continuing to raise T past that point actively hurts accuracy rather than helping it.

Page 17 of 20

Manual vs. automatic seeding. The automatic seed (darkest point after a light blur) lands on a true 0-intensity pixel inside the pupil on all 3 images. The manual seed (crop centre) is intensity 1 on good_lighting and intensity 3 on glasses_reflection – close enough that its Medium-threshold region size is within 1% of the auto-seed result on both (e.g. glasses_reflection: 8,777px manual vs. 8,770px auto). But on poor_lighting the manual seed sits on an intensity-54 pixel (the pupil is measurably off-centre in that crop) and its Medium- T region (7,002px) is over 5x larger than the auto-seed result (1,352px), because it starts from a brighter running mean and immediately incorporates part of the iris/sclera boundary that the correctly-centred auto seed does not yet reach at the same T (Figure 7, rightmost panel). Automatic darkest-point seeding is measurably more reliable here than assuming the pupil is centred.

4- vs 8-connectivity. The size difference is small in absolute terms (0-252px, at most 2.8% of the 4-connected count) but not zero, and not one-directional: 8-connectivity picks up 252/4/91 diagonal-reachable pixels the 4-connected run misses on the 3 images respectively, while 4-connectivity separately keeps 72/4/65 pixels that the 8-connected run's earlier diagonal absorptions end up excluding once its running mean has shifted – visible as both colours of speckle in the Figure 5/8/11 XOR maps, not a one-sided fringe. Shape compactness is measurably lower for 8-connectivity on 2 of 3 images (0.526 → 0.415 on good_lighting, 0.609 → 0.566 on poor_lighting; unchanged at 0.325 on glasses_reflection), i.e. the extra diagonal reach makes the boundary slightly more jagged rather than rounder. Processing time is consistently higher for 8-connectivity (1.42x-1.47x, mean 1.45x, across 30 timed repeats per image) – expected, since every popped pixel tests 8 candidate neighbours instead of 4 – but the absolute cost difference (0.4-2.6ms/call) is irrelevant at this image scale.

Region splitting and merging. Splitting alone produces 250-256 leaves on all 3 images (nearly all blocks split down to the 20px minimum rather than stopping early on homogeneity – histogram equalization raises local contrast/noise enough that few 20px+ blocks satisfy the max-min ≤ 25 test before hitting the size floor). Merging collapses this by 5.4x-19.2x: most aggressively on good_lighting (250 → 13 regions), whose large, evenly-lit sclera/skin background merges into very few big regions, and least on glasses_reflection (256 → 47 regions), whose glare hotspot, eyelid crease and reflection streak fragment the intensity landscape into many small mean-distinct patches that never satisfy the merge tolerance with their neighbours (visible in Figure 12's much more fragmented colour map compared to Figure 6's).

Honest limitation. The reference pupil mask used for missing/leakage/accuracy (src.segment.segment_eye's Otsu-anchored dark-core mask) is itself an approximation, not ground truth – Task 6 already documents that dataset-wide Otsu-derived masks on these crops can include eyebrow/eyelash strips alongside the pupil. This is visible directly in Figure 7 (poor_lighting): the region-growing mask at High threshold visually connects into the eyelid crease above the pupil, yet the leakage column still reads 0 at that threshold, because the reference mask's own dark-core envelope already extends into that same crease. Accuracy numbers here should be read as

"recovery of the same dark-core envelope Task 6 anchors on", not as pupil-only ground-truth accuracy.

Page 18 of 20

Limitations & Future Work

- **Seed sensitivity.** Automatic darkest-point seeding is reliable on all 3 test images here, but remains a single-pixel decision with no fallback. The manual centred seed on `poor_lighting` (intensity 54, Seeds table) sat outside the pupil and produced a Medium-threshold region over 5x larger than the correctly-seeded auto run (7,002px vs. 1,352px, Observations) purely because it started from a much brighter running mean – a single bad seed silently produces a badly wrong region with no signal in the output that anything went wrong. *Future work:* seed validation (e.g. reject seeds far from the image's own darkest-percentile pixels) or multi-seed voting.
- **No sub-pixel boundary.** Every region boundary here is a set of whole pixels – a pixel is either in the mask or not, with no partial-membership or sub-pixel edge estimate – so reported areas carry an inherent boundary-perimeter quantization error that matters more for a small pupil than a large one.
- **Fixed global T.** A single threshold is applied uniformly across the whole image regardless of local contrast – unlike Task 6's adaptive threshold, which recomputes its cutoff per neighbourhood specifically to handle this (Comparison table above). *Future work:* a locally-scaled T, e.g. proportional to a local contrast estimate around the current frontier.
- **Blocky, quadtree-aligned boundaries.** `split_merge`'s regions are unions of axis-aligned rectangular leaves (Figures 6/9/12), so even after merging, region boundaries are staircase-shaped along the leaf grid rather than following the pupil/iris's typically curved true boundary – visible directly in the merged-region colour maps.
- **Merge scalability.** The implemented adjacency test in `MERGE` is $O(L^2)$ in the leaf count `L` (Theory & Formal Background – Complexity analysis); a smaller `min_size` (finer quadtree, better boundary fidelity) would grow `L` and make this the dominant cost. *Future work:* a spatially-indexed adjacency query in place of the pairwise scan.

Conclusion

Seeded region growing with a continuously-updated region mean was implemented in `src/region_growing.py` and evaluated on the same 3 dataset images used in Tasks 3 and 6, using an automatic darkest-point seed (verified more reliable than a naive centred manual seed, especially on the off-centre pupil in `poor_lighting`) and 4 threshold levels. Region size grows with threshold on every image but the growth is non-linear, with a measured leakage-onset point (past the reference Otsu-anchored pupil mask) between the High and Very High thresholds on `good_lighting/poor_lighting`, and no leakage at all within the tested range on `glasses_reflection`, where the IR glare boundary constrains growth. **8-connectivity** is the better default for this application: it recovers a small but real set of diagonal- adjacent pupil-boundary pixels (up to 2.8% more coverage) that 4-connectivity's orthogonal-only steps miss, at a processing-time cost (mean 1.45x slower, low single-digit milliseconds either way) that is immaterial at this image scale – though the comparison also showed neither connectivity is strictly a superset of the other, since the running-mean update makes the two BFS traversals diverge slightly rather than merely differ by a fixed diagonal fringe. **Region splitting and merging** (quadtree split + union-find merge) was implemented as a second region-based technique in the same module: splitting alone produced 250-256 leaves on all 3 images (most blocks hit the 20px size floor rather than stopping early on homogeneity), and merging reduced this by 5.4x-19.2x, least aggressively on `glasses_reflection` whose glare/reflection/eyelid-crease intensity landscape resists merging into large uniform regions. All quantitative claims in this report are grounded in the measured CSVs (`results/task7_threshold_metrics.csv`, `results/task7_connectivity_metrics.csv`, `results/task7_splitmerge_metrics.csv`), not visual impression alone. The formal treatment in Theory & Formal Background traces the 4- vs 8-connectivity non-superset

result to a specific, provable cause – the running-mean similarity criterion's order-dependence – rather than leaving it as an unexplained empirical curiosity, and the failure-mode analysis in Analysis & Discussion grounds both the `poor_lighting` leak at $T=60$ and the absence of any leak on `glasses_reflection` in the same intensity-profile mechanism (Figure 19): a shallow gradient the running mean can climb versus a hard barrier it cannot cross.